CS2201 Computer Organization & Architecture

L12: MIPS Pipeline for Multi-Cycle Operations



John Jose

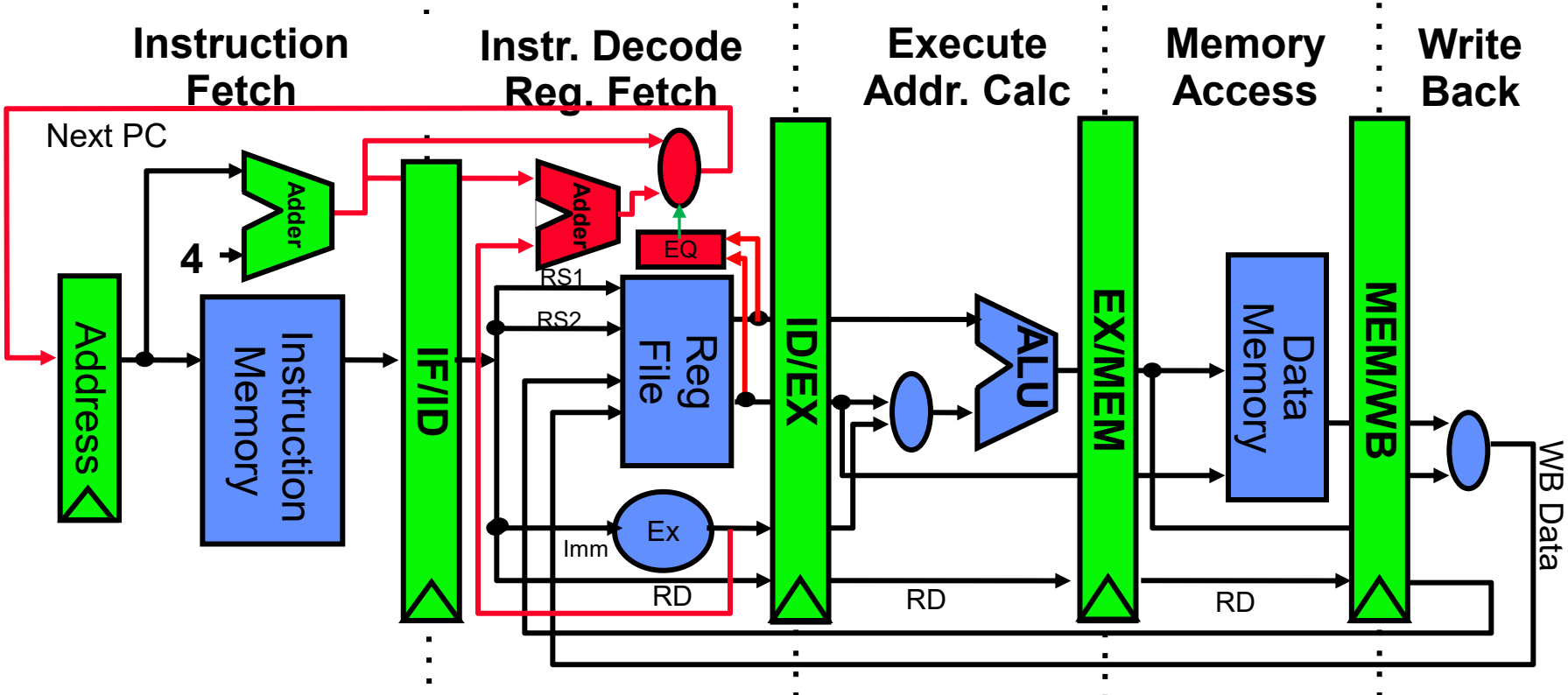
Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Branch Optimized MIPS Pipeline

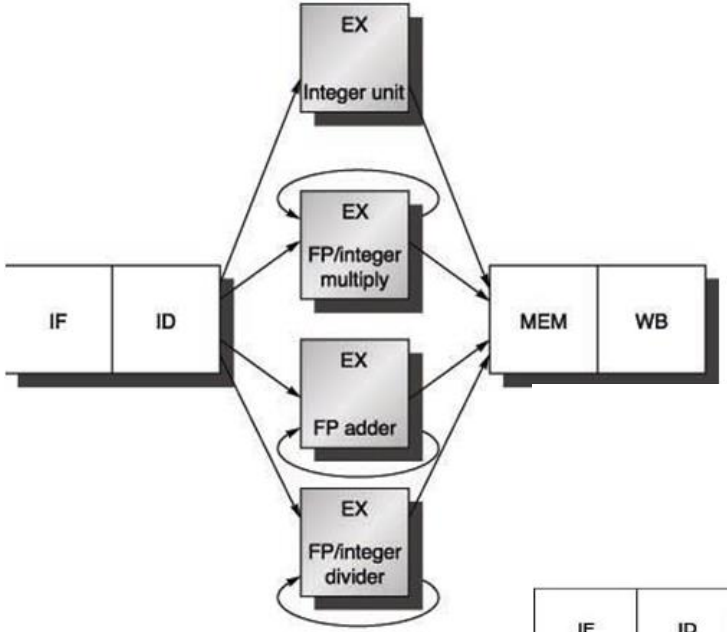
- Can EXE stage complete the operation in 1 cycle always?

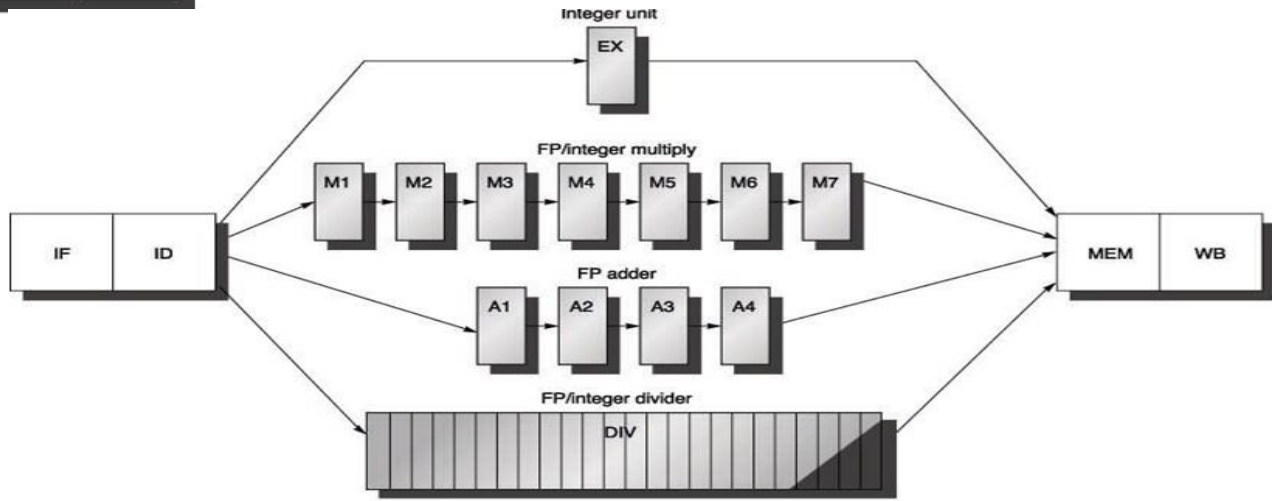


Multi-cycle Operations

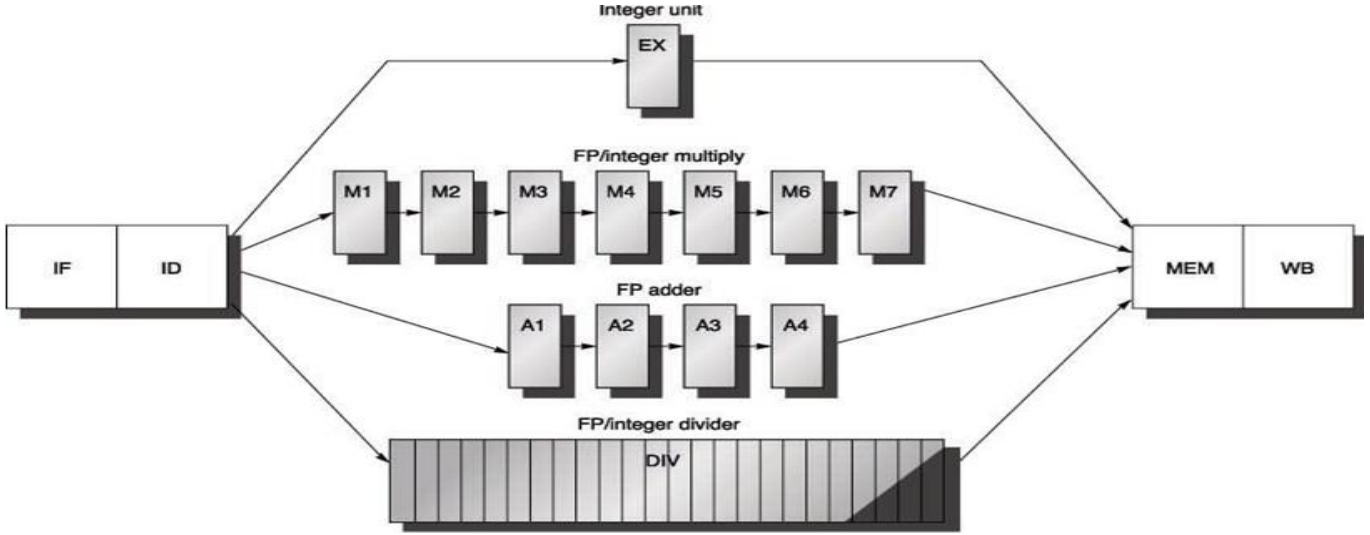
- Can EXE stage complete the operation in 1 cycle?
- Some operations require more than 1 clock cycle to complete.
  - Floating Point/Integer Multiply
  - Floating Point/Integer Divide
  - Floating Point Add/Sub
- Dedicated hardware units are available on the processor for performing these operations.

Multi-cycle Operations





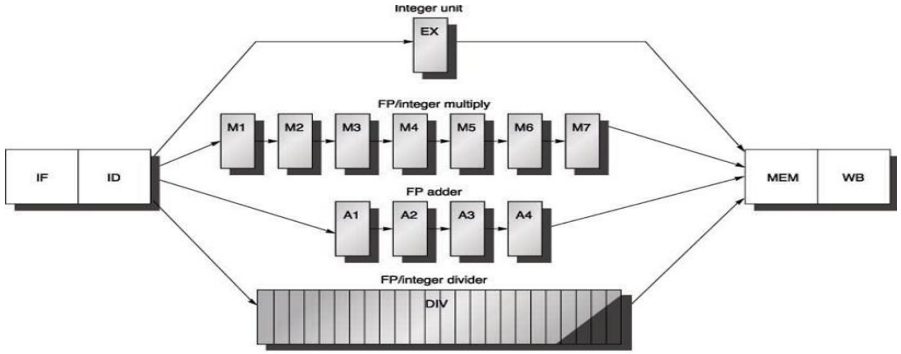
Multi-cycle Operations



Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Latencies and initiation intervals for functional units.

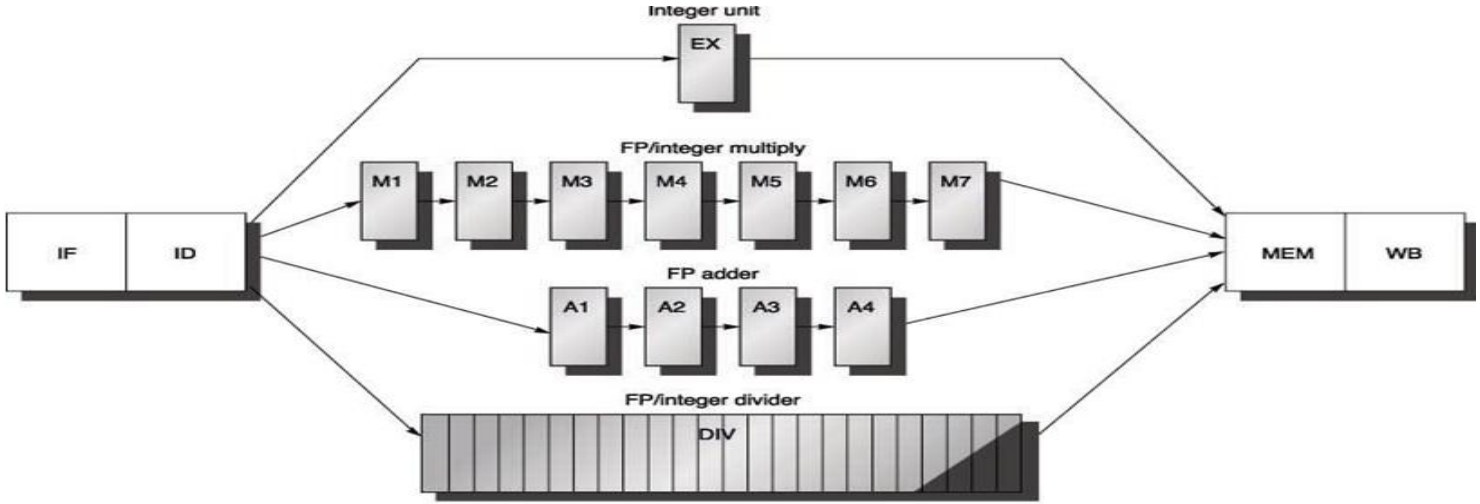
Multi-cycle Operations



Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

- **Latency:** The number of intervening cycles between an instruction that produces a result and an instruction that uses the result.
- **Initiation / Repeat Interval:** The number of cycles that must elapse between issuing two operations of a given type.

Multi-cycle Operations



MUL.D	IF	ID	*M1*	M2	M3	M4	M5	M6	M7	MEM	WB
ADD.D		IF	ID	*A1*	A2	A3	A4	**MEM**	WB		
L.D			IF	ID	*EX*	**MEM**	WB				
S.D				IF	ID	*EX*	*MEM*	WB			

The pipeline timing of a set of independent FP operations. The stages in italics show where data are needed, while the stages in bold show where a result is available. The ".D" extension on the instruction mnemonic indicates double-precision (64-bit) floating-point operations. FP loads and stores use a 64-bit path to memory so that the pipelining timing is just like an integer load or store.

Issues in Longer Latency Pipelines

- Since divide unit is not-pipelined – structural hazard
- Instruction have varying runtimes–more register writes/cycle
- WAW hazards possible
- Stalls for RAW hazards will be more.

	Clock cycle number																
Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L.D F4,0(R2)	IF	ID	EX	MEM	WB												
MUL.D F0,F4,F6		IF	ID	stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
ADD.D F2,F0,F8			IF	stall	ID	stall	stall	stall	stall	stall	stall	A1	A2	A3	A4	MEM	WB
S.D F2,0(R2)					IF	stall	stall	stall	stall	stall	stall	ID	EX	stall	stall	stall	MEM

Issues in Longer Latency Pipelines

	Clock cycle number										
Instruction	1	2	3	4	5	6	7	8	9	10	11
MUL.D F0,F4,F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
ADD.D F2,F4,F6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
L.D F2,0(R2)							IF	ID	EX	MEM	WB

- Single write port (Serialize completion) vs multiple write ports
- Resolve write port conflicts in ID stage and stall issue by 1
- Stall either of the instruction (priority basis) at MEM / WB stage
- Stall at MEM will force a stall to trickle at EX, M7, A4 stages

Issues in Longer Latency Pipelines

	Clock cycle number										
Instruction	1	2	3	4	5	6	7	8	9	10	11
MUL.D F0,F4,F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
ADD.D F2,F4,F6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
L.D F2,0(R2)						IF	ID	EX	MEM	WB	

- WAW hazard at register F2
- Delay issue (ID→ EX) of L.D until ADD.D enters MEM stage
- Keep the result of ADD.D and give it to needed instruction.
- Hence only L.D will write on F2.

How to handle Issues in Longer Pipelines

- **Check for structural hazard** in DIV.D and write ports
- **Check for RAW data hazard at ID stage**: If the source of an instruction in ID is Fi then Fi should be there as the name of destination of instruction in ID/A1, A1/A2, A2/A3 and ID/M1, M1/M2,.... M6/M7
- **Check for WAW data hazard**: If any instruction in A1,…A4, M1,..M7 has the same destination as the instruction in ID and the time at which they reach WB is same, delay issue by 1 cycle and repeat.
- **Perform operand forwarding** from EX/MEM, A4/MEM, M7/MEM, D/MEM, MEM/WB

Name Dependence & Output dependence

- Two instructions use the same name but no flow of information.
- Not a true data dependence, but is a problem when reordering instructions
  - **Antidependence:** instruction j writes a register or memory location that instruction i reads
    - Initial ordering (i before j) must be preserved
  - **Output dependence:** instruction i and instruction j write the same register or memory location
    - Ordering must be preserved
- To resolve, use renaming techniques

Control Dependence

- Ordering of instruction with respect to a branch instruction
  - Instruction that is control dependent on a branch **cannot be moved before the branch** so that its execution is no longer controller by the branch
  - An instruction that is **not** control dependent on a branch **cannot be moved after the branch** so that its execution is controlled by the branch.

```
if p1 {
        S1;
};
if p2 {
        S2;
}
```

Control Dependence

- Instruction that is control dependent on a branch cannot be moved **before** the branch so that its execution is no longer controller by the branch
- An instruction that is not control dependent on a branch cannot be moved **after** the branch so that its execution is controlled by the branch.

Example 1:

```
    DADDU R1,R2,R3
    BEQZ R2,L1
    LW R1, 0(R2)
L1: …
```

Example 2:

```
    DADDU R1,R2,R3
    BEQZ R4,skip
    DSUBU R1,R5,R6
skip:
    OR R7,R1, R8
```



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/